

Tour Planner

Final Project Protocol

Angular Frontend · ASP.NET Core Backend · PostgreSQL · OpenRouteService · Leaflet

Student	Danial Hassani, Theodor Gentschev
Personal identifier	Matrikelnummer: 52402501, Matrikelnummer: 52503664
Submission	Final Hand-In
Date	05 July 2026
Repository link	https://github.com/GEN22693/FH-University-TourPlanner.git

Short summary: Tour Planner is a full-stack route-planning application. Users can register, log in, create tours, inspect calculated routes on a Leaflet map, add tour logs, search across tours and logs, import/export data, and see computed statistics.

Table of Contents

1. Project Overview
2. Requirement Coverage
3. Final Feature Set
4. Application Architecture
5. Data Model and Class Structure
6. Use Cases and Sequence Flow
7. UX Concept and Wireframes
8. Implementation Details
9. Security, Validation and Configuration
10. Testing Decisions and Test Flow
11. Unique Feature
12. Design Patterns and Library Decisions
13. Lessons Learned
14. Tracked Time
15. How to Run the Project
16. Appendix

1. Project Overview

Tour Planner is a personal route planning and tour history application. The final version is no longer a frontend-only prototype. The application now uses an Angular frontend, an ASP.NET Core backend, a PostgreSQL database and external route calculation through OpenRouteService. The map is rendered in the browser with Leaflet and OpenStreetMap tiles.

The intermediate version stored users, tours and logs in browser LocalStorage. For the final submission, this was replaced by a real backend connection. The frontend now communicates with REST API endpoints via Angular HttpClient. Authentication is handled with JWT tokens, and protected routes are guarded in Angular.

- Main user flow: Register -> Login -> Tours -> Tour Detail -> Map -> Logs.
- Main backend flow: Controller -> Business Service -> Repository -> PostgreSQL.
- Main external integration: OpenRouteService calculates distance, duration and route geometry; Leaflet visualizes the route.

2. Requirement Coverage

The following table maps the final implementation to the expected final hand-in checklist. I used this section as a practical self-check before submission.

Requirement area	Implemented evidence
C# / Java backend	ASP.NET Core Web API in C#, split into API, Business, Data, Models and Tests projects.
Angular frontend	Angular standalone components, Angular signals, router, guards, interceptor and services.
MVVM UI	HTML templates are the View, component classes act as ViewModels, models/services provide application data and API communication.
Layer-based architecture	API controllers call business services; business services use repositories and external clients; repositories use ApplicationDbContext.
Design pattern	Repository Pattern, Facade Pattern for the map, Dependency Injection, DTO pattern, Guard/Interceptor pattern.
PostgreSQL + ORM	Tour data is stored in PostgreSQL through Entity Framework Core and migrations.
No SQL injection	No raw SQL is used in the normal data access flow; EF Core LINQ and parameterized queries are used.
Configuration	Connection string is stored in appsettings; OpenRouteService API key is provided through user secrets/configuration, not hardcoded.
OpenRouteService + Leaflet	OpenRouteService returns route distance, duration and GeoJSON geometry; Leaflet renders it on the map.
Tour CRUD	Create, read, update and delete tours through /api/tours endpoints.
TourLog CRUD	Create, read, update and delete tour logs through nested /api/tours/{tourId}/logs endpoints.
Computed attributes	Popularity and child-friendliness are calculated from tour logs and returned in TourResponseDto.
Full-text search	Backend search endpoint searches tour fields, tour log fields and computed values.

Import / Export	JSON export downloads all user tour data; JSON import creates tours and nested logs.
Unique feature	Statistics dashboard: total tours, total logs, total distance, estimated time, average rating, most popular and best rated tour.
Protocol	Architecture, use cases, sequence diagram, UX, library decisions, design patterns, testing decisions, unique feature and time tracking are documented here.

3. Final Feature Set

- User registration and login with backend authentication.
- JWT token storage on the frontend for the active session.
- Auth interceptor that automatically attaches the token to protected requests.
- Auth guard that protects /tours and /tours/:id from anonymous access.
- Tour overview with backend-loaded tour list, statistics, search, import, export and create action.
- Tour detail page with all important tour attributes, computed attributes, Leaflet route map and tour logs.
- Tour logs with create, edit, delete, rating, difficulty, distance and time values.
- Backend search across tour fields, tour log fields and computed attributes.
- JSON import and export for tour data.
- Statistics dashboard as the additional unique feature.
- Centralized CSS design system with reusable UI classes for cards, buttons, forms, tables, modals, navigation and map.

4. Application Architecture

The application is organized as a layered full-stack application. This keeps the responsibilities separated and makes the code easier to explain and maintain.

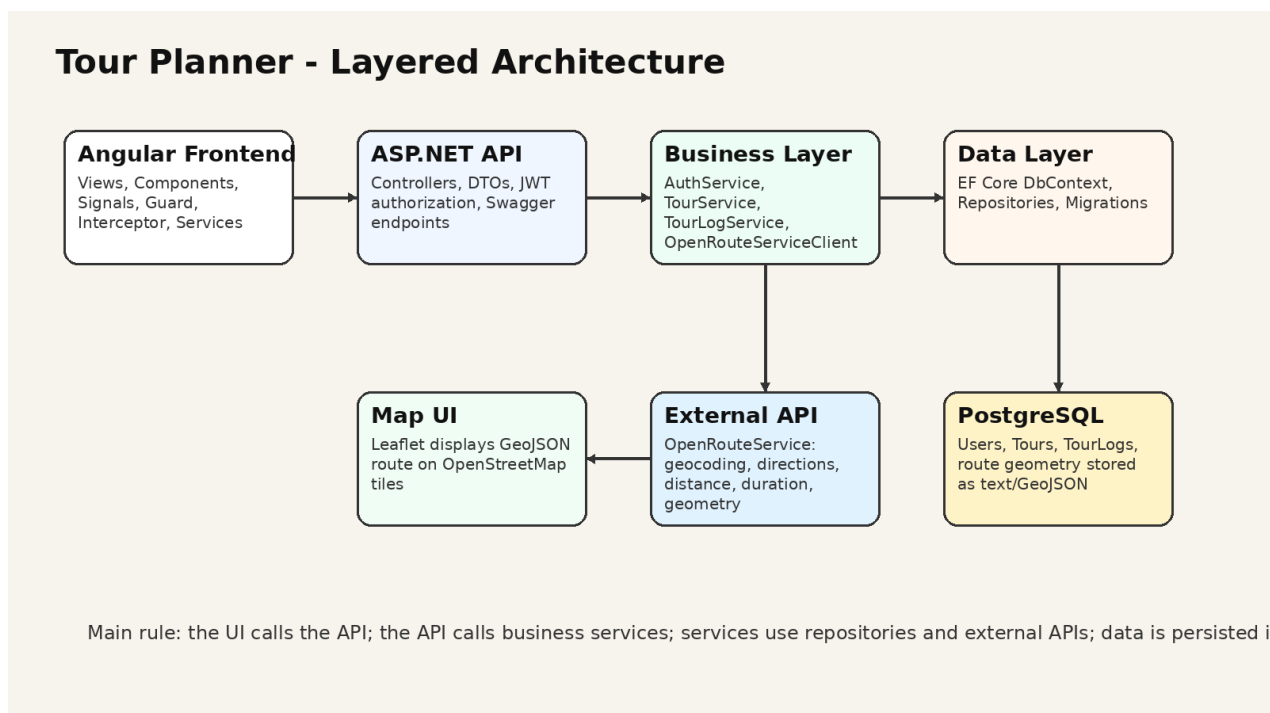


Figure 1: Layered architecture of the final Tour Planner application.

4.1 Frontend Architecture

The frontend is built with Angular standalone components. The most important Angular elements are:

- Pages: login, register, tour list and tour detail.
- Shared components: navigation and route map.
- Services: AuthService, TourService and MapFacadeService.
- Models: Auth, Tour and TourLog interfaces.
- Guard: auth.guard.ts protects private pages.
- Interceptor: auth.interceptor.ts attaches the JWT token automatically.

I refactored the UI so that the HTML mainly describes the structure and Angular bindings, while the global CSS file contains reusable design classes. This makes the final code much easier to read than a page full of repeated utility classes.

4.2 Backend Architecture

The backend is structured into separate projects. The API project exposes endpoints, the Business project contains the application logic, the Data project contains EF Core context and repositories, and the Models project contains entities, DTOs and enums.

Backend project	Main responsibility
TourPlanner.Api	Controllers, Program.cs, authentication setup, Swagger, HTTP endpoints.
TourPlanner.Business	AuthService, TourService, TourLogService, OpenRouteServiceClient, business interfaces.
TourPlanner.Data	AppDbContext, repositories, EF Core migrations.
TourPlanner.Models	Entities, DTOs and enums shared across backend layers.
TourPlanner.Tests	Unit test project for backend behavior.

5. Data Model and Class Structure

Domain Model and Main DTOs

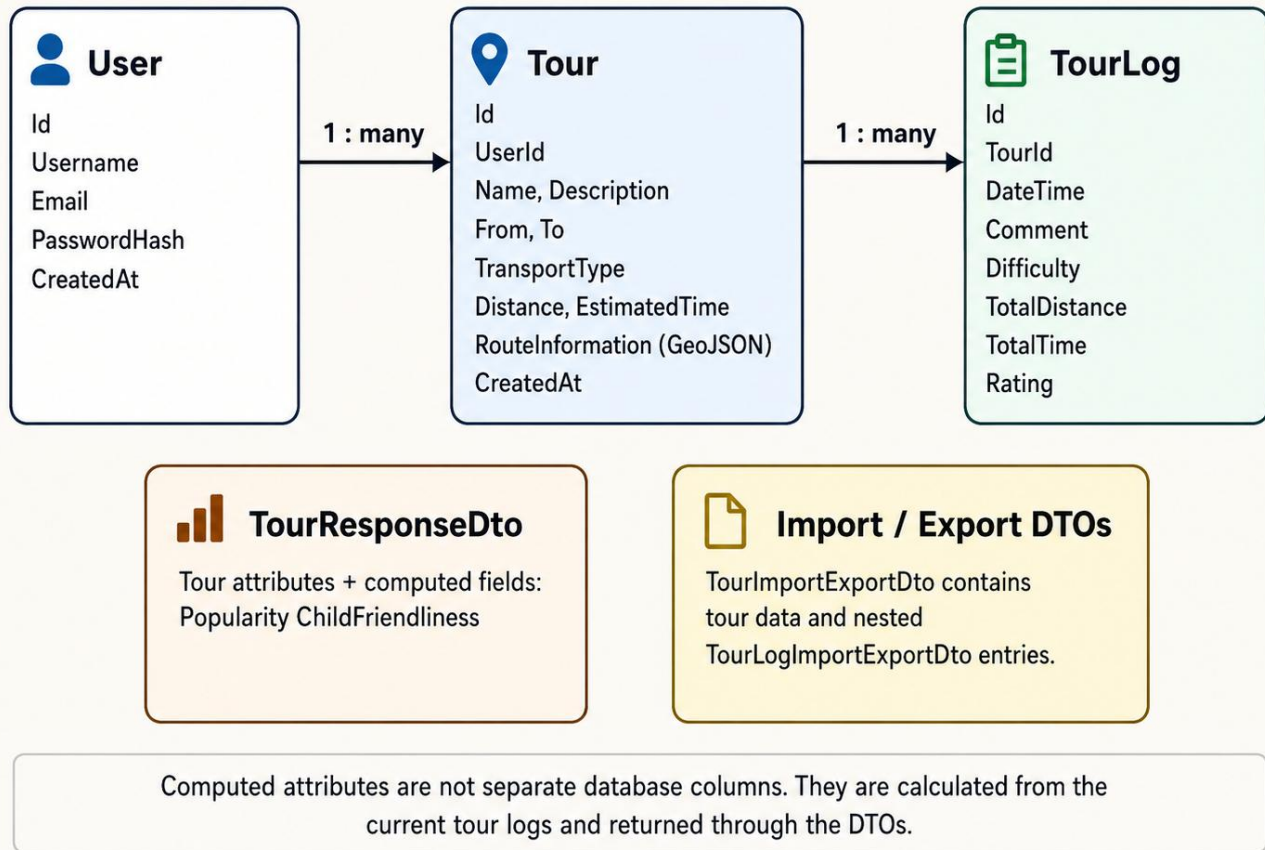


Figure 2: Main domain classes and DTO structure.

The main data model contains three core entities: User, Tour and TourLog. A user can own many tours, and a tour can have many tour logs. The backend uses DTOs to return only the needed data to the frontend and to keep API contracts separate from database entities.

5.1 Main Entities

Entity	Important fields	Relationship
User	Id, Username, Email, PasswordHash, CreatedAt	One user has many tours.
Tour	Id, UserId, Name, Description, From, To, TransportType, Distance, EstimatedTime, RouteInformation, CreatedAt	One tour belongs to one user and has many logs.
TourLog	Id, TourId, DateTime, Comment, Difficulty, TotalDistance, TotalTime, Rating	One log belongs to one tour.

5.2 RouteInformation and Database Migration

Originally, RouteInformation stored a short readable text. For the final map, it now stores route geometry as a GeoJSON LineString. This is why the database column was changed from varchar(4000) to text using the migration MakeRouteInformationText. Without this change, long route geometries caused a PostgreSQL value-too-long error.

6. Use Cases and Sequence Flow

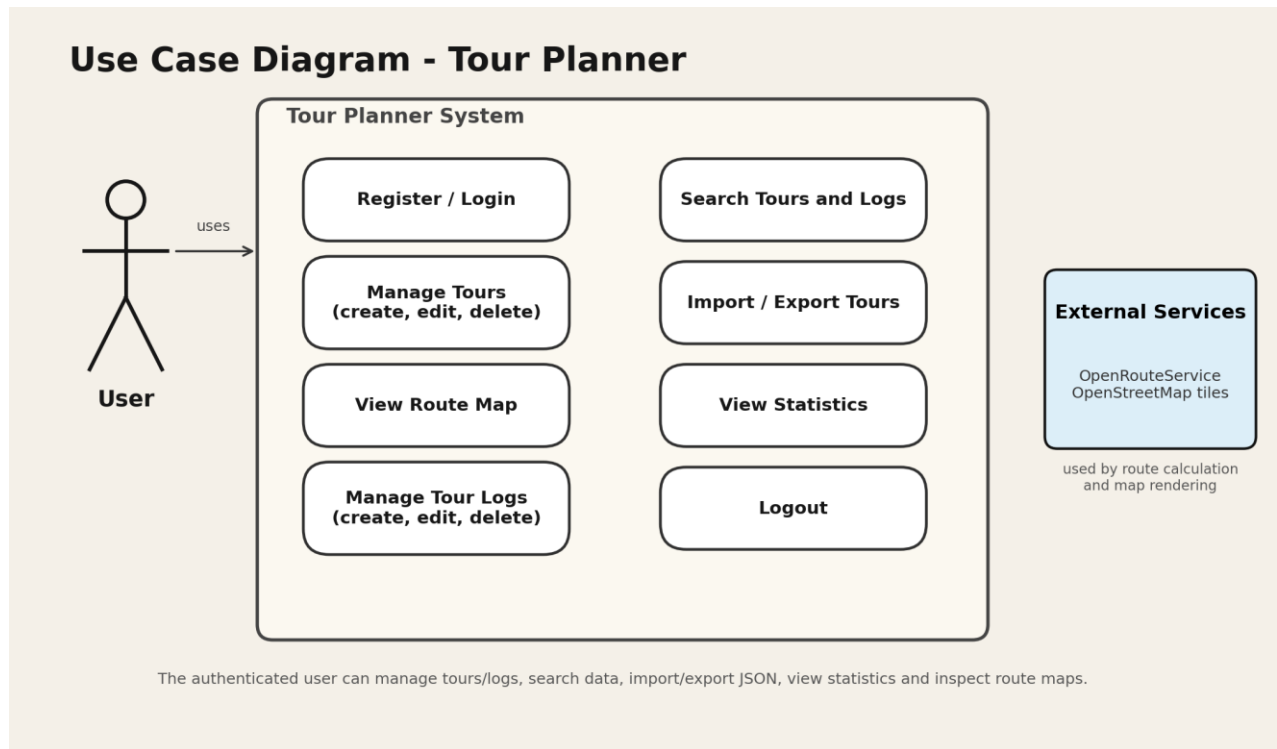


Figure 3: Main user use cases in the final Tour Planner application.

6.1 Main Use Cases

Use case	Description
Register account	The user creates a new account with username, email and password.
Login	The user logs in and receives a JWT token from the backend.
Manage tours	The user creates, opens, edits and deletes tours.
Inspect route map	The user opens a tour detail page and sees the calculated route on a Leaflet map.
Manage logs	The user adds, edits and deletes tour logs after completing a route.
Search	The user searches in tour data, tour logs and computed values.
Import/export	The user exports a JSON file or imports existing tour data.
View statistics	The user sees summary values about all tours and logs.

The required sequence diagram focuses on the full-text search flow. The frontend sends the query to the backend, the backend loads tours and logs for the current user, computes popularity and child-friendliness, and returns only matching tours.

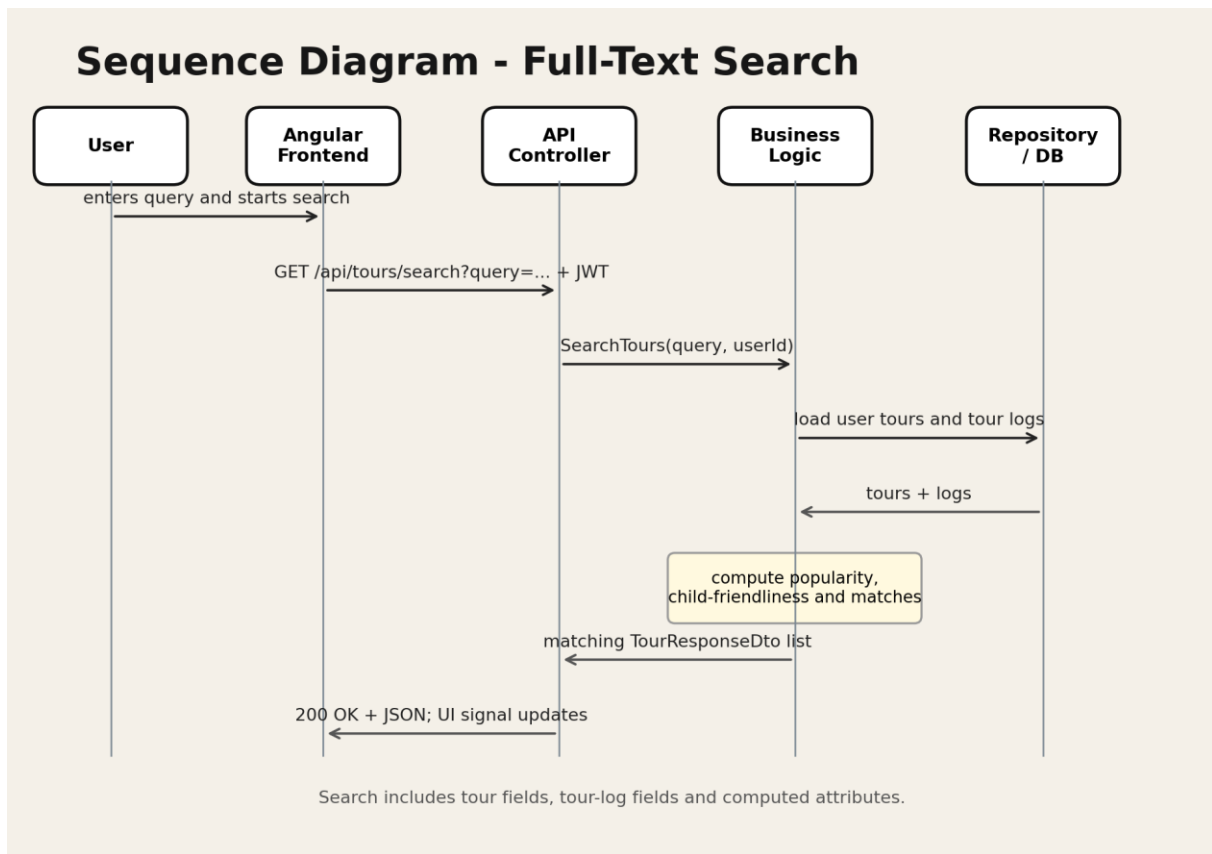


Figure 4: Sequence flow for full-text search including tour logs and computed attributes.

7. UX Concept and Wireframes

The UX concept is minimal and calm. The interface uses a large title style, light background gradients, compact data cards and a fixed navigation. The goal is to keep route management focused without overwhelming the user.

The intermediate mockup document defined the key screens and UX goals: login, register, tour overview, tour detail, map/log flow and empty states. The final implementation keeps the same core flow but adds backend data, statistics, search, import/export and computed attributes.

Screen	UX goal
Login / Register	Fast entry and simple onboarding with consistent visual language.
Tour overview	Compact route library with quick access to search, create, import and export.
Tour detail	Show selected route attributes once, then focus on map and logs.
Map and logs	Move naturally from route inspection to logging.
Empty states	Tell the user what is missing and what action is available next.

UX NOTES

Purpose

1. Saved tours are shown in a compact table-like list.
2. Search and New action are placed near the route library title.
3. UX goal: users can quickly find, open or delete a route.

Figure 5: Intermediate wireframe - tour overview page.

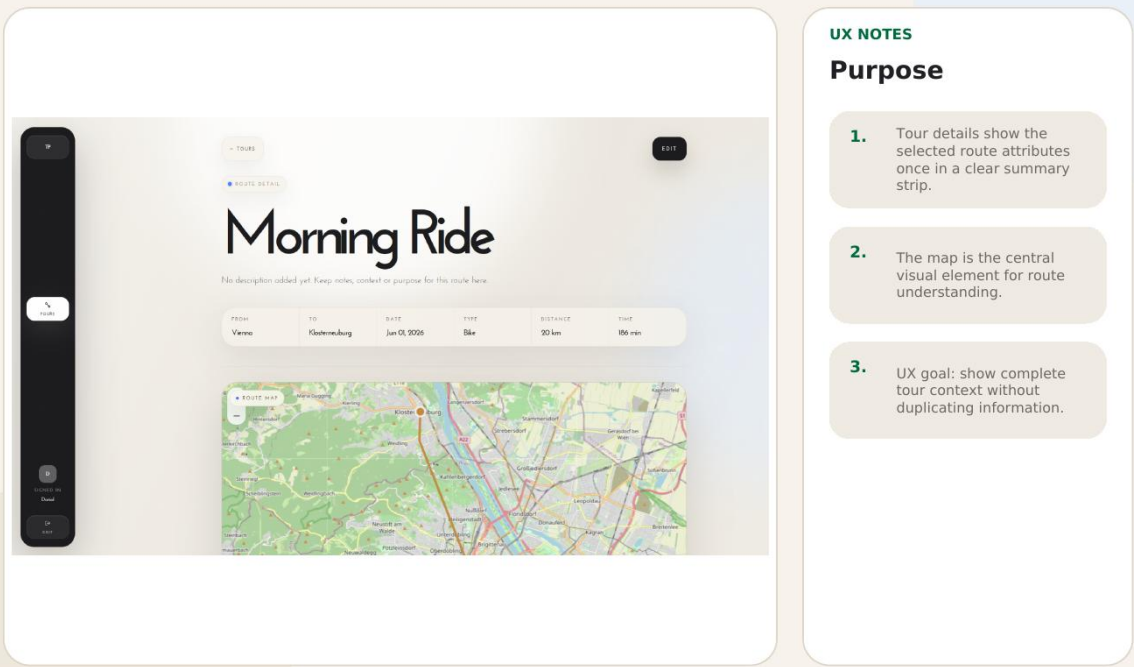


Figure 6: Intermediate wireframe - tour detail page with map.

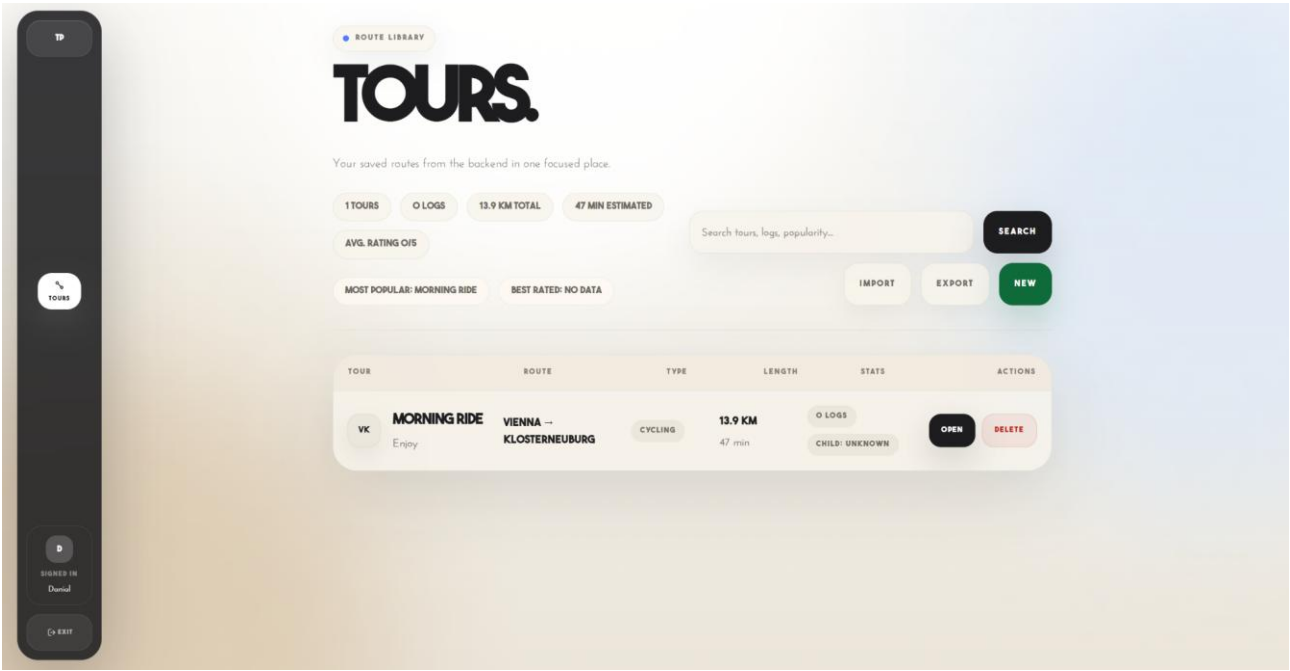


Figure 7: Final tour overview with statistics, search, import, export and new tour action.

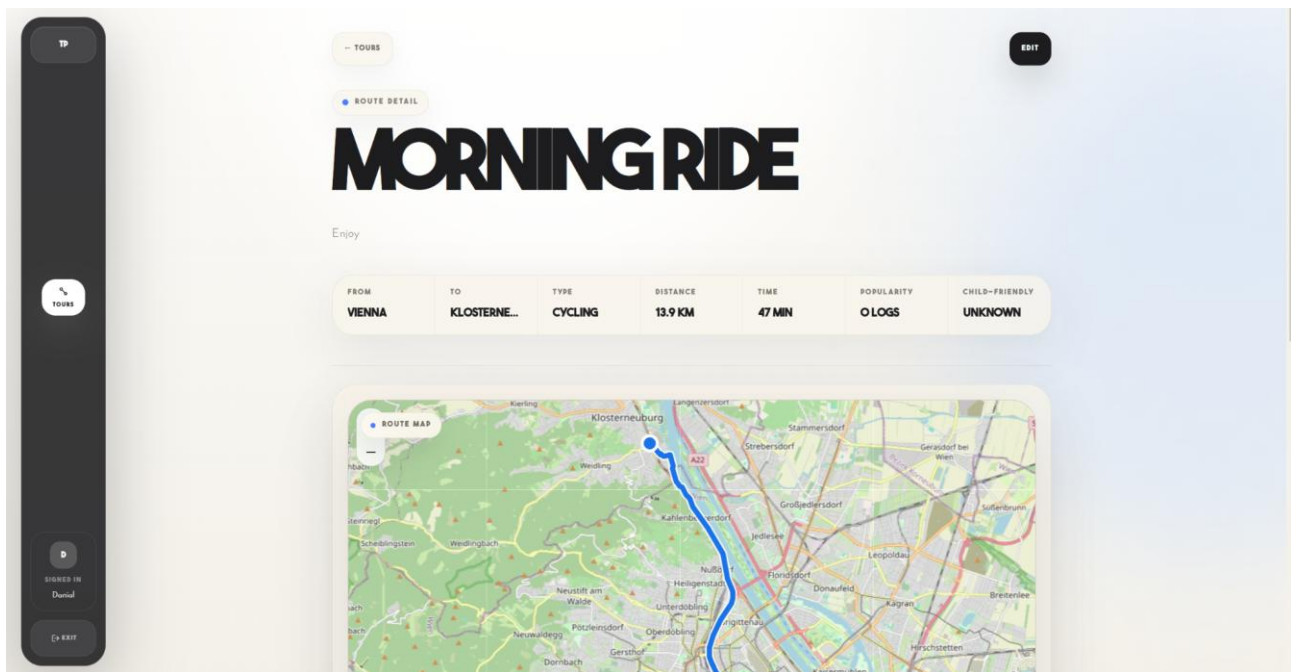


Figure 8: Final tour detail page with route summary and map.

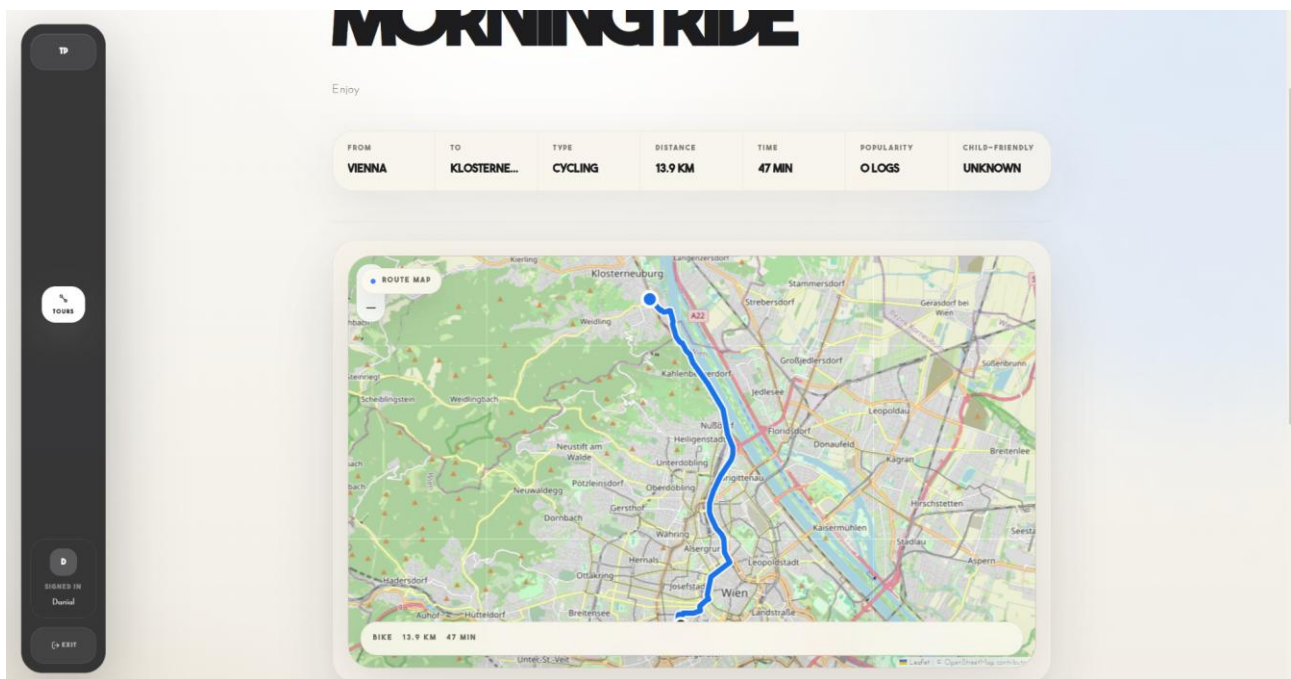


Figure 9: Final map display with real route geometry from OpenRouteService.

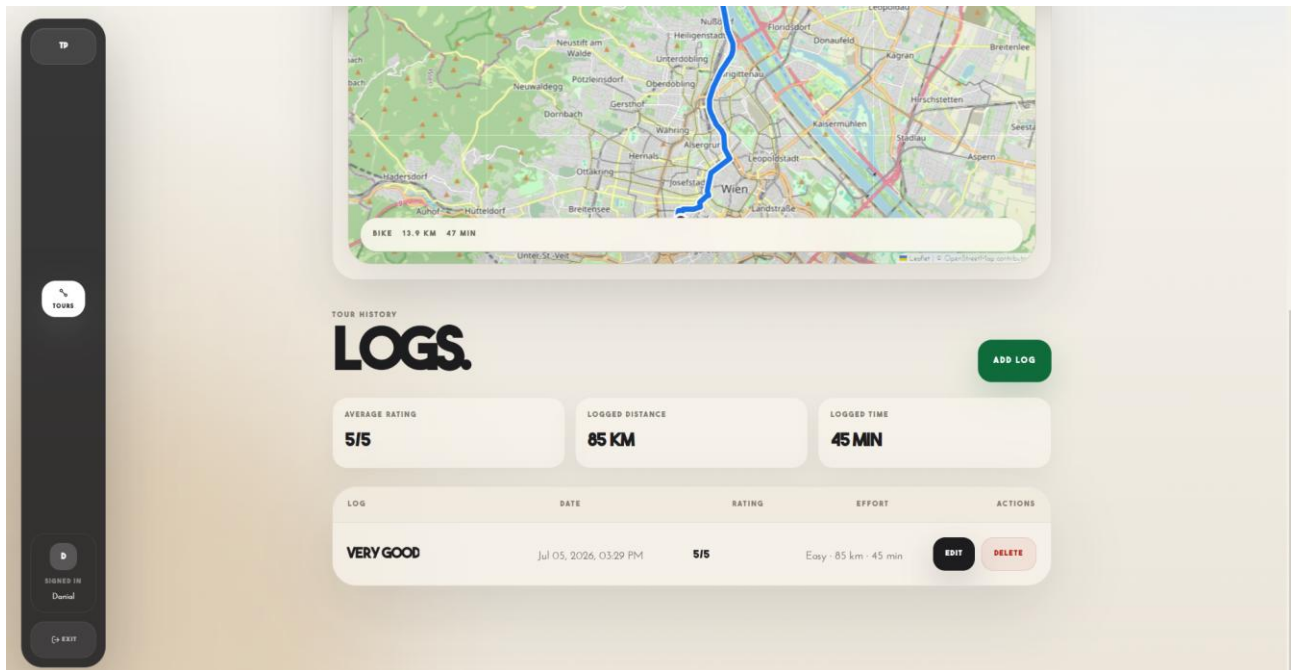


Figure 10: Final tour log section with statistics and log actions.

8. Implementation Details

8.1 Authentication

Authentication is implemented with backend endpoints for register, login and me. The frontend AuthService sends login/register requests with HttpClient. After a successful login, the returned JWT token is stored for the browser session. The AuthInterceptor automatically attaches the token as a Bearer token to API requests, and the AuthGuard redirects unauthenticated users back to /login.

8.2 Tour Management

Tours are created, edited, loaded and deleted through the /api/tours endpoints. When a tour is created or updated, the backend calls OpenRouteService to calculate the route. The backend saves distance, estimated time and route geometry. The frontend displays distance in kilometers and formats backend TimeSpan values into readable minutes/hours.

8.3 Route Map

The RouteMap component receives from, to, transport type, distance, estimated time and routeInformation. The MapFacadeService initializes Leaflet, parses GeoJSON from routeInformation and draws the route as a polyline. If the backend geometry cannot be parsed, the service still has a fallback strategy, but the final goal is to use backend route geometry.

8.4 Tour Logs

Tour logs are nested under a selected tour. Each log has a date/time, comment, difficulty, total distance, total time and rating. The frontend converts user-entered minutes into TimeSpan format before sending data to the backend. The detail page shows average rating, total logged distance and total logged time.

8.5 Full-Text Search

Search is implemented as a backend endpoint: GET /api/tours/search?query=.... The endpoint searches across tour fields, log fields and computed attributes. The frontend keeps a search input in the tour overview page and

displays the resulting list from the backend. This keeps the search user-scoped and avoids filtering only the data currently visible in the browser.

8.6 Import and Export

Export uses GET /api/tours/export and downloads a JSON file. The file includes tour data and nested log data. Import uses POST /api/tours/import and sends a JSON array to the backend. During import, the backend creates the tours and logs again and recalculates route information based on the stored start and destination values.

8.7 API Endpoint Overview

Endpoint	Purpose
POST /api/auth/register	Create account.
POST /api/auth/login	Authenticate and return JWT token.
GET /api/auth/me	Return the current authenticated user.
GET /api/tours	Return all user tours.
POST /api/tours	Create tour and calculate route.
GET /api/tours/search	Search in tours, logs and computed attributes.
GET /api/tours/statistics	Return the statistics dashboard data.
GET /api/tours/export	Download tour data as JSON.
POST /api/tours/import	Import tours and logs from JSON.
GET /api/tours/{id}	Return one tour.
PUT /api/tours/{id}	Update one tour.
DELETE /api/tours/{id}	Delete one tour.
GET /api/tours/{tourId}/logs	Return logs for a tour.
POST /api/tours/{tourId}/logs	Create log.
PUT /api/tours/{tourId}/logs/{logId}	Update log.
DELETE /api/tours/{tourId}/logs/{logId}	Delete log.

9. Security, Validation and Configuration

- JWT authentication protects tour and log endpoints. The backend extracts the user id from the token claims so users only access their own data.
- The frontend validates required fields before sending requests. The backend also returns controlled error messages for invalid input, missing routes or import problems.
- Entity Framework Core is used instead of raw SQL for the normal persistence flow, reducing SQL injection risk.
- The database connection string is stored in configuration. The OpenRouteService API key is configured through user secrets/local configuration and is not hardcoded in the Angular frontend.
- RouteInformation was migrated to a text column because GeoJSON route geometry can be longer than 4000 characters.
- The application uses Swagger during development to verify all endpoints and DevTools Network to inspect actual frontend/backend communication.

10. Testing Decisions and Test Flow

Testing was split into automated backend unit tests and manual end-to-end checks in the browser. The automated tests are implemented with NUnit and Moq in the TourPlanner.Tests project and focus on the service layer, because this is where the most important validation rules, route handling and user-scoped behavior are located. I also tested the full running application manually, because several final requirements depend on the integration between Angular, the ASP.NET API, PostgreSQL and OpenRouteService.

Test area	What was tested
Authentication	Register, login, logout, protected routes and automatic JWT token usage through the Angular guard and interceptor.
Tour CRUD	Tour creation, loading, editing, deletion and persistence after browser reload.
TourLog CRUD	Tour-log creation, editing, deletion and persistence after reload.
Map and route geometry	Leaflet displays the route using GeoJSON geometry returned from OpenRouteService and stored in RouteInformation.
Search	Search terms match tour fields, log comments, difficulty values and computed values such as child-friendliness.
Import/Export	Export downloads a JSON file; import reads a JSON array and creates tours with nested logs again.
Computed attributes	Popularity changes when logs are added or deleted; child-friendliness is calculated from log difficulty, distance and time.
Responsive UI	The fixed desktop sidebar and the main content layout were checked with the final overview and detail screens.
Build checks	Frontend: ng build. Backend: dotnet build and dotnet test.
TourServiceTests.cs	NUnit/Moq tests cover missing tour name, missing start/destination, identical start and destination, mapping of returned tours, missing tour handling, delete result handling and route data returned from a mocked route service.
TourLogServiceTests.cs	NUnit/Moq tests cover missing tour handling, rating validation, negative distance, zero time, successful log creation and log deletion behavior.

10.1 Manual Final Test Flow

1. Start PostgreSQL, run the EF Core database update and start the ASP.NET backend.
2. Start the Angular frontend and open <http://localhost:4200>.
3. Register a new user and log in.
4. Create a tour from Vienna to Klosterneuburg and check the calculated distance and time.
5. Open the tour detail page and verify that the real route is visible on the Leaflet map.
6. Add a tour log with difficulty, distance, time and rating.
7. Verify that popularity, average rating and log statistics update.
8. Search for text from tour data and log data.
9. Export the tours to JSON and re-import the file.
10. Edit and delete a tour and a log.
11. Log out and confirm that protected pages redirect to login.

11. Unique Feature: Statistics Dashboard

The mandatory unique feature is a statistics dashboard on the tour overview page. It summarizes the user's route library and gives quick insight before opening individual tours.

- Total number of tours.
- Total number of logs.
- Total route distance.
- Total estimated time.
- Average rating across logs.
- Most popular tour based on log count.
- Best rated tour based on average rating.

This feature is implemented through the backend endpoint GET /api/tours/statistics and displayed at the top of the Angular tour overview page.

12. Design Patterns and Library Decisions

12.1 Implemented Design Patterns

Pattern	Where it appears	Why it is useful
Repository Pattern	TourRepository, TourLogRepository, UserRepository	Separates database access from business logic.
Facade Pattern	MapFacadeService	Hides Leaflet setup, drawing and cleanup behind a small service API.
DTO Pattern	CreateTourDto, TourResponseDto, Import/Export DTOs	Keeps API contracts separate from database entities.
Dependency Injection	ASP.NET services and Angular inject()	Keeps dependencies explicit and testable.
Interceptor Pattern	auth.interceptor.ts	Adds JWT token automatically to outgoing API requests.
Guard Pattern	auth.guard.ts	Protects private routes on the frontend.

12.2 Library Decisions

Library / Technology	Decision reason
Angular	Required frontend framework; good support for standalone components, routing, signals and reusable services.
TypeScript	Adds type safety to frontend models, API data and component logic.
ASP.NET Core Web API	Modern C# backend, clear controller structure, Swagger support and JWT integration.
Entity Framework Core	ORM for PostgreSQL, migrations and LINQ-based database access.
PostgreSQL	Persistent relational storage for users, tours and logs.
OpenRouteService	External API for geocoding, route directions, distance, duration and geometry.
Leaflet	Lightweight map library for displaying route geometry in the browser.
OpenStreetMap tiles	Map tile source used through Leaflet.

Swagger	Manual endpoint testing and API documentation during development.
CSS design system	Final UI uses reusable CSS classes instead of repeating long styling directly in each template.

13. Lessons Learned

- German locale bug: `double.ToString()` uses a comma as decimal separator on German Windows. OpenRouteService rejected coordinates such as 16,348. I fixed this with `CultureInfo.InvariantCulture`.
- ORS response format: The Directions API returns GeoJSON under `features[].properties.summary` and `features[].geometry`, not a simple JSON object. I discovered this by logging and inspecting the raw HTTP response.
- User-scoping at repository level: Filtering by `userId` inside EF Core WHERE clauses is safer than only checking in the service. This way a developer cannot accidentally skip the ownership check in a new endpoint.
- Route geometry storage: A short text field was not enough after storing the real GeoJSON route. The database column `RouteInformation` had to be migrated from `varchar(4000)` to text.
- Frontend maintainability: Moving repeated design code into reusable CSS classes made the HTML easier to read, test and present.

14. Tracked Time

Task	Approx. hours
Requirement analysis and planning	6
Project setup and database configuration	8
Backend architecture and EF Core setup	10
Authentication and JWT implementation	10
Tour CRUD backend and frontend integration	12
TourLog CRUD backend and frontend integration	10
OpenRouteService integration and route geometry	12
Leaflet map integration and route display	9
Search feature across tours and logs	7
Import/export implementation	7
Statistics and computed attributes	8
Frontend UI design and CSS refactoring	12
Testing, debugging and final fixes	12
Protocol, diagrams and final documentation	7
Total	120

15. How to Run the Project

15.1 Backend

From the backend folder:

```
dotnet restore .\TourPlanner.sln  
  
dotnet ef database update --project .\TourPlanner.Data\TourPlanner.Data.csproj --startup-  
project .\TourPlanner.Api\TourPlanner.Api.csproj --context AppDbContext  
  
dotnet run --project .\TourPlanner.Api\TourPlanner.Api.csproj --launch-profile http
```

Backend URL: <http://localhost:5161>. Swagger: <http://localhost:5161/swagger>.

15.2 Frontend

```
npm install  
  
ng serve
```

Frontend URL: <http://localhost:4200>.

15.3 Final Build Commands

```
ng build  
  
dotnet build .\TourPlanner.sln  
  
dotnet test .\TourPlanner.sln
```

16. Appendix

16.1 Final Project Structure Evidence

The final Visual Studio project contains the API, Business, Data, Models and Tests projects. The frontend contains reusable models, services, pages and shared components.

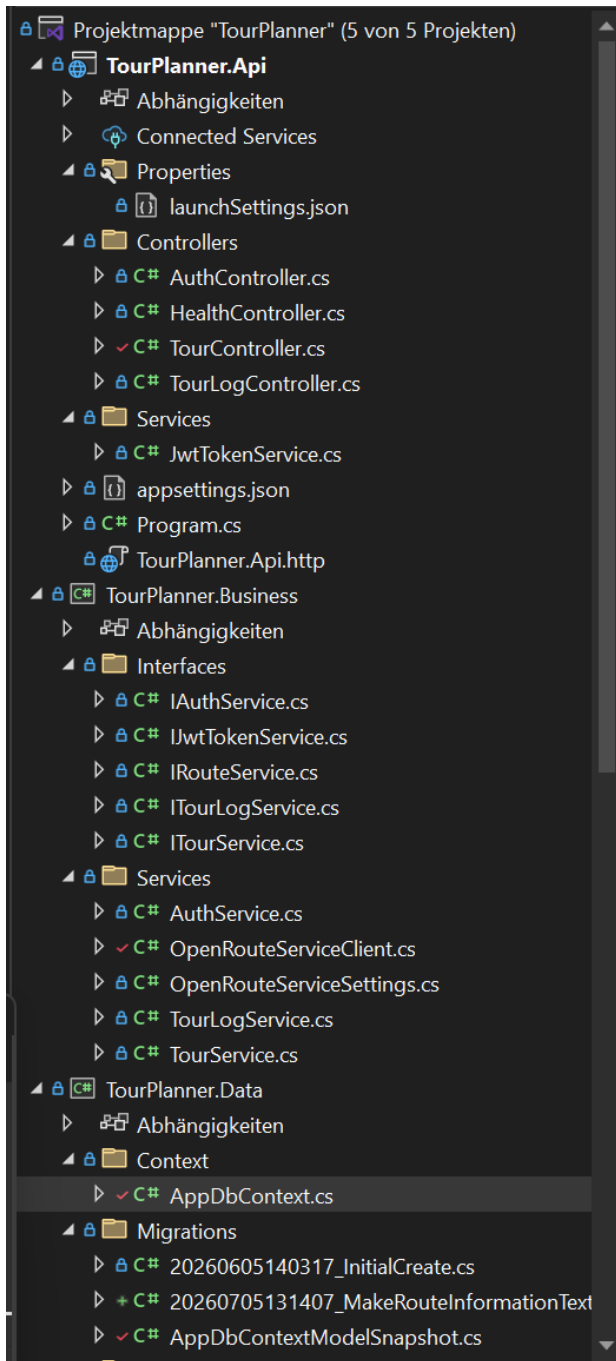


Figure 11: Visual Studio project structure - API, Business and Data layers.

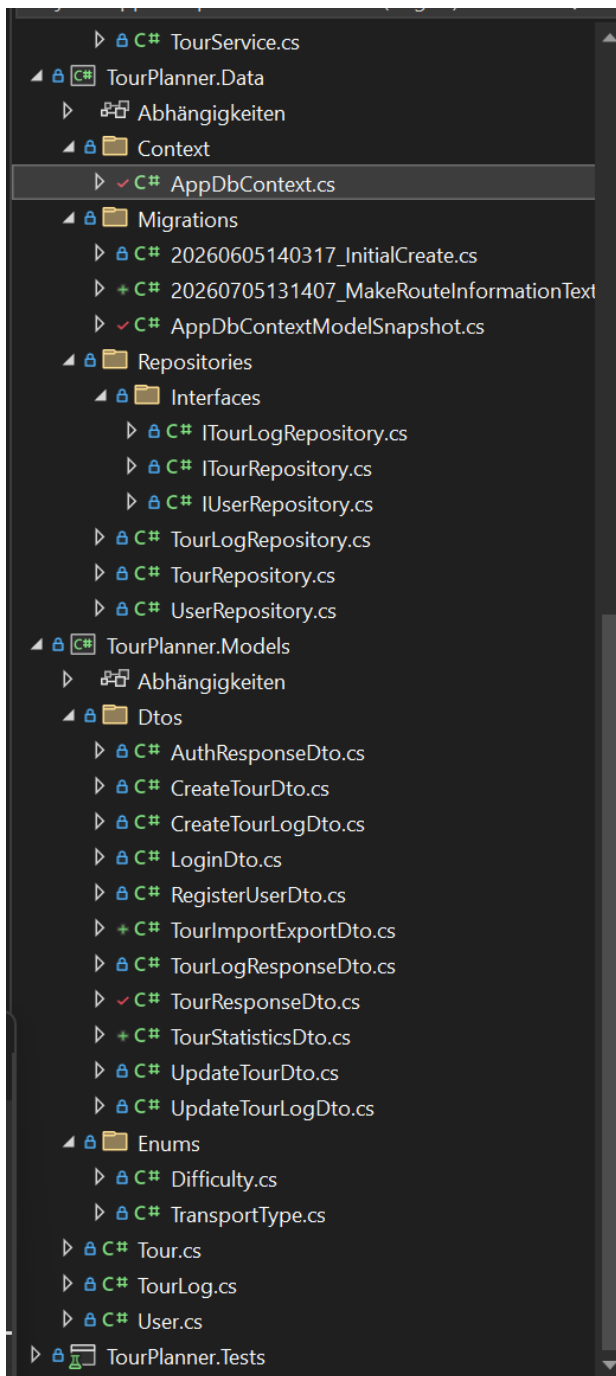


Figure 12: Visual Studio project structure - DTOs, enums and models.

16.2 External Sources and Tools Used

- Angular documentation for standalone components, signals, router, forms, guards and interceptors.
- ASP.NET Core and Entity Framework Core documentation for Web API, dependency injection, migrations and PostgreSQL integration.
- Leaflet documentation for map initialization and polylines.
- OpenRouteService documentation for geocoding and route directions.
- OpenStreetMap tiles through Leaflet.
- Swagger for backend endpoint testing.
- Browser DevTools Network tab for checking API responses.

16.3 Git Repository

Git repository link: <https://github.com/GEN22693/FH-University-TourPlanner.git>